



FPGA NGS 端到端加速论文分析与实现路线图

一、文档目的

基于论文 [A 28nm Fully Integrated End-to-End Genome Analysis Accelerator for Next-Generation Sequencing](#) 进行整理，目标是回答两个问题：

1. 论文的逻辑结构和实现方法是什么。
2. 我们是否可以在 FPGA 上实现论文中的端到端算法，以及应当如何落地。

二、核心结论

可以在 **FPGA** 上实现论文中的端到端算法，但不能仅凭论文无歧义复刻作者的 **28nm ASIC 芯片**。

更准确地说：

1. 从算法层面看，可以实现。
2. 从工程系统层面看，可以做出功能上等价的 **FPGA 原型**。
3. 从芯片级复现层面看，论文信息不足，无法精确复刻全部微结构细节。
4. 从性能层面看，**FPGA** 可以实现同类流水线，但通常不能等价达到论文中的 **ASIC 面积、频率和能效**。

因此，合理目标不是“逐门级复刻论文芯片”，而是“以 **FPGA** 实现同一条端到端 **NGS** 分析流水，并尽量保留其关键算法与架构思想”。

三、论文的逻辑结构

概括为四层递进。

3.1 问题定义

作者针对 NGS 数据分析提出一个完整硬件化目标：

1. short-read mapping
2. haplotype calling
3. variant calling
4. genotyping

论文认为，既有硬件工作大多只覆盖其中一部分，缺少真正的端到端整合；而如果不同步骤分散在不同平台上执行，会产生很高的数据交换开销。

3.2 算法 workflow 定义

Section II 定义了完整算法链：

1. 用 FM-index 做 seed exact match。
2. 用 Smith-Waterman 对候选位置做 inexact match。
3. 对映射结果做 paired-end 约束和 rescue。
4. 基于 de Bruijn graph 做 haplotype assembly。
5. 用 S-W backtracking 做 variant discovery。
6. 用 Viterbi + Pair-HMM 做 genotype likelihood 计算。

3.3 硬件架构映射

Section III 将上述算法映射成硬件模块：

1. 两个 aligner engines 负责 short-read mapping。
2. 一个 caller engine 负责 haplotype calling、variant calling、genotyping。
3. 通过 system bus 和双缓冲 DRAM 维持端到端流水。

3.4 性能验证与边界说明

Section IV/V 给出 28nm ASIC 的测试结果，并明确说明了系统边界：

1. 最大支持 read length 为 150。
2. Smith-Waterman 评分参数硬编码。
3. pipeline 固定，不支持用户自由重配置。

4. 不支持第三代测序长读长 °
-

四 、论文真正实现的端到端算法链

4.1 Short-read mapping

目标是从 reads 中找出它们在 reference genome 中最可信的位置 °

流程如下：

1. 将 read 切成 seeds °
2. 利用 FM-index 找出与 seed 精确匹配的候选位置 °
3. 对候选参考片段做相似度计算 °
4. 若为 paired-end ’则联合考虑两端 read 的配对位置和分数 °
5. 若不能正确配对 ’则启动 rescue °

4.2 Haplotype calling

目标是在局部区域内 ’从已经映射好的 reads 重建可能的 haplotypes °

流程如下：

1. 提取局部 reads 和 reference segment °
2. 切分为 k-mers °
3. 构建 de Bruijn graph °
4. 若出现 loop ’则增大 k-mer 长度 °
5. 通过深度优先搜索组装 haplotypes °

4.3 Variant calling

目标是找出 haplotypes 与 reference 的差异 °

流程如下：

1. 对 haplotype 与 reference 再做一次 Smith-Waterman °
2. 记录方向矩阵 °

3. 从最高分位置开始 backtracking °
4. 输出 SNP 和 INDEL 等变异候选 °

4.4 Genotyping

目标是计算每个变异位点的 genotype likelihood ’并输出 PL 值 °

流程如下 :

1. 针对 read-haplotype 对 ’使用 Viterbi + Pair-HMM 计算 likelihood °
 2. 构成 read-to-haplotype likelihood matrix °
 3. 聚合为 read-to-allele likelihood °
 4. 进一步形成 genotype likelihood °
 5. 最终转为 Phred-scaled Likelihood (PL) °
-

五 、关键实现方法分析

5.1 FM-index exact matching

论文采用 FM-index 进行 seed exact match ’其核心操作包括 :

1. LF-mapping
2. SA query

作者使用 two-stage LF-mapping :

1. 第一阶段用 M-base suffix lookup table ’一次性缩小搜索范围 °
2. 第二阶段对剩余字符继续进行 LF-mapping °

这说明作者在 exact matching 阶段已经明显做了面向硬件的迭代压缩 ’减少完全串行 LF 查询的开销 °

5.2 Inexact matching: Smith-Waterman

论文使用带 affine gap 的 Smith-Waterman :

- $E(i, j)$ 表示 insertion 分数
- $F(i, j)$ 表示 deletion 分数
- $S(i, j)$ 表示 alignment/mismatch 分数

关键点：

1. 评分规则是预定义的。
2. 选择得分矩阵中的最高分作为 similarity score。
3. paired-end 时，两个 reads 的结果需要联合评估。

这意味着短读长比对的核心仍然是动态规划，而不是简单编辑距离近似。

5.3 Rapid Similarity Calculator (RSC)

这是论文中最重要的硬件优化之一。

问题背景：

- 如果每个 candidate 都跑完整 S-W，成本太高。

RSC 的作用：

1. 先快速判断 candidate 是否几乎和 reference 一致。
2. 支持识别单碱基 mismatch。
3. 支持头尾最多 5 个 bases 的 soft-clip 检查。
4. 依据 BWA-MEM 的评分逻辑筛掉不需要进入 S-W 的候选。

论文给出的效果：

1. 节省超过 70% 的 S-W 计算。
2. 单个 RSC 8 cycles 处理一个 read。
3. 在其目标场景下保持与 BWA-MEM 一致的打分行为。

从工程角度看，RSC 的意义非常大，因为它把最重的 DP 工作量前移过滤了。

5.4 Pair and Rescue

论文强调 paired-end 不是附加功能，而是精度提升的关键机制。

Pairing 逻辑：

1. 两端 reads 要落在合理距离内 °
2. 正确配对结果优先于单端局部最优结果 °

Rescue 逻辑 :

1. 若 R1 和 R2 没有 properly paired ,则触发 rescue °
2. 在 mate 附近约 2000bp 范围重新取 reference segment °
3. 对未正确对齐的一端重新对齐 °

论文声称 :

1. false negative 降低 36.3% °
2. rescue 的额外 DP 代价约占总时间 0.4% °

这一点说明作者不是单纯追求吞吐 ,而是明确在用硬件结构换整体精度和 sensitivity °

5.5 Parallel k-mer Processing Engine (PKPE)

这是 haplotype calling 的核心硬件模块 °

其要解决的问题是 :

1. de Bruijn graph 构建涉及大量状态更新 °
2. graph assembly 存在顺序依赖 °

论文方案 :

1. 使用 32 个 Graph Arrays (GA) °
2. 每个 GA 包含 128 个 Graph Cells (GC) °
3. 总计 2048 个 GCs °
4. GC 支持三种模式 :Graph Construction `Graph Update `Assembly °
5. k-mer 长度支持 16 到 48 ,并可动态调整 °

其本质是用专门的数据结构阵列替代通用排序/共享计算结构 ,以换取图构建和遍历速度 °

5.6 Variant Discovery Engine

变异发现阶段使用 S-W 与 backtracking 识别 haplotype 相对 reference 的差异 °

这部分算法上相对直接 :

1. 先计算 score matrices °
2. 再用 direction matrices 回溯路径 °
3. 提取 SNP/INDEL 等变异 °

需要注意的是 ’论文虽然给了算法路径 ’但对 Variant Discovery Engine 的微结构描述明显少于 RSC `PKPE `GLCE ’因此这里存在实现细节缺口 °

5.7 Genotype Likelihood Computing Engine (GLCE)

这部分是 genotyping 的核心模块 °

组成 :

1. Viterbi Decoding Engine (VDE)
2. Max Likelihood Finder (MLF)
3. PL Calculator (PLC)
4. ping-pong buffers

论文关键做法 :

1. 将概率运算转到 log-domain °
2. 将乘法转化为加法 ’降低硬件成本 °
3. 使用 16-bit 预定义 log 概率 °
4. VDE 由 4 个 processing element arrays 组成 °

这一设计很适合 FPGA ’因为它本质上就是定点 `规则 `可流水的动态规划阵列 °

六 `系统架构的真正创新点

论文的价值不只是“把几个模块硬件化” ’而是让整条链路真正能跑起来 °

6.1 双 aligner + 单 caller

系统由两个 aligner engines 和一个 caller engine 组成 °

这样做的原因是 :

1. mapping 本身最耗时 °
2. caller 侧也包含 graph 构建 `variant discovery 和 genotyping` 不轻 °
3. 通过前后段并行`可以提高端到端吞吐` °

6.2 Ping-pong DRAM

作者把 DRAM 分成两块：

1. 一块供 aligner 使用 °
2. 一块供 caller 使用 °

这样当 caller 在处理样本 A 时`aligner 可以同时处理样本 B`避免流水线空转 °

6.3 Paired-end simultaneous alignment

作者明确指出`如果 paired-end 的两个 reads 不同步处理`会在等待 pair 完整结果时产生严重 stall °

所以其设计策略是：

1. 两个 read 同时对齐 °
2. 一边算一边评估 °
3. 若已经找到 full score`则尽早终止无意义计算` °

6.4 真正瓶颈仍是 DP

论文实验明确指出`瓶颈不在 DRAM`而在动态规划计算 °

这件事非常重要`因为它说明：

1. 端到端系统的成败主要取决于 DP 阵列设计 °
 2. FPGA 落地时最该优化的不是 PCIe 或 DDR 带宽`而是 mapping 和 genotyping 中的 DP 并行度与流水深度 °
-

七、我们是否可以在 **FPGA** 中实现论文中的端到端算法

7.1 为什么可以

原因有四个。

7.1.1 算法具备硬件友好性

论文中的核心计算都适合硬件流水实现：

1. FM-index 查询
2. Smith-Waterman DP
3. backtracking
4. de Bruijn graph 更新
5. Viterbi/Pair-HMM

这些操作要么是规则阵列计算，要么是受限状态机与存储更新，不依赖复杂动态控制流。

7.1.2 作者已经做了硬件化裁剪

例如：

1. read 长度固定到 150。
2. S-W 评分固定。
3. Viterbi 参数预定义并量化。
4. RSC 用于大规模过滤 S-W。
5. k-mer 长度范围被限制在 16 到 48。

这说明论文不是在讲“任意 NGS 软件算法上 FPGA”，而是在讲“专门裁剪过的硬件版端到端 pipeline”。

7.1.3 论文原型本来就与 **FPGA** 板级存储协同

论文实验环境里，ASIC 芯片挂在 PCB 上，并连接到 AMD-Xilinx VCU118，利用板上 DDR4 存储 reads、FM-index 和 reference。

更关键的是，论文明确说明：

1. FPGA 本身不做计算 °
2. FPGA 主要负责数据与控制转发 °

这从侧面说明其数据组织方式本来就适合板级系统 ’而不是必须依赖完全定制 SoC 环境 °

7.1.4 瓶颈不在 **DRAM** ’而在 **DP**

如果瓶颈在外存接口 ’那么 FPGA 会更难做 ;但论文指出瓶颈主要在 DP ’这意味着只要 PE 阵列和流水线设计得当 ’FPGA 原型是可行的 °

7.2 为什么说不能简单等价复刻

7.2.1 论文细节不够完整

论文没有给出 :

1. 完整 RTL 级模块接口 °
2. 所有 buffer 深度和时序约束 °
3. 所有异常路径和边界处理逻辑 °
4. Variant Discovery Engine 的完整微结构 °
5. scheduler `candidate pairer `quality calculator 的实现细节 °

因此 ’想只凭论文直接复原作者芯片 ’会有大量实现空白 °

7.2.2 FPGA 与 ASIC 物理约束不同

ASIC 的优势在于 :

1. 可定制 SRAM 与寄存器结构 °
2. 更高布线自由度 °
3. 更低单位逻辑能耗 °
4. 更易做高密度 PE 阵列 °

而 FPGA 受限于 :

1. LUT/FF/BRAM/URAM 的固定结构 °
2. 布线拥塞 °
3. 时钟频率更低 °
4. 大规模随机访存更难优化 °

所以功能可复现 ’不代表代价可复现 °

八 `FPGA 实现时最难的地方

8.1 FM-index 的访存模式

LF-mapping 和 SA 查询涉及不完全规则的访问模式 °放到 FPGA 上 ’问题不在算法是否成立 ’而在 :

1. 如何组织 OCC/SA/lookup table °
2. 如何减少 DDR 随机访问 °
3. 如何做缓存和 burst 访问 °

8.2 Smith-Waterman 的资源消耗

即使有 RSC ’mapping 阶段剩余的 S-W 仍是大头 °其挑战包括 :

1. PE 阵列数量与资源之间的折中 °
2. 候选并行度与时钟的折中 °
3. paired-end 与 rescue 引入的调度复杂度 °

8.3 PKPE 的状态存储与图遍历

de Bruijn graph 相关逻辑会消耗大量存储和控制资源 °尤其在 FPGA 上 ’Graph Cells 如何映射到 BRAM/URAM ’是一个核心架构问题 °

8.4 GLCE 的流水平衡

Genotyping 阶段虽然更规则 ’但要注意 :

1. 位宽是否足够 °
2. log-domain 定点误差是否可接受 °
3. ping-pong buffer 的吞吐能否接住 caller 上游输出 °

8.5 端到端调度和回压

最难的通常不是某个单独模块，而是：

1. 前后级吞吐不匹配。
2. 某些样本或区域复杂度很高。
3. rescue、graph loop、backtracking 等路径导致局部长尾延迟。

因此必须从一开始就设计好回压、队列和缓冲策略。

九、FPGA 实现路线图

下面给出一版现实可行的 FPGA 落地路线，而不是理想化的一步到位方案。

9.1 总体原则

建议遵循三个原则：

1. 先做功能闭环，再做性能优化。
2. 先做 mapping 和 genotyping 两个 DP 核心，再做 graph 和端到端调度。
3. 先做论文同构近似系统，再决定是否追求更激进的并行度。

9.2 第一阶段：建立最小可运行原型

目标：先跑通从输入 reads 到最终 genotype/PL 输出的完整流程。

建议保留的模块：

1. FM-index exact match
2. 简化版 Smith-Waterman PE array
3. paired-end 评分
4. rescue 基本流程
5. 简化版 de Bruijn graph
6. S-W backtracking variant discovery
7. Viterbi/Pair-HMM likelihood

建议简化的内容：

1. aligner engine 先只做一个，不要一开始就做两个。
2. caller engine 先串行执行 haplotype `variant` `genotype`，不要一开始就追求完全重叠。
3. PKPE 初版可以先用较少数量的 graph cells。
4. k-mer 长度先固定一到两个值，再加动态调整。
5. 先在较小数据集或局部区域验证正确性。

这一阶段的目标不是吞吐，而是证明：

1. 算法链跑通。
2. 中间结果一致。
3. 定点化不会使结果完全失真。

9.3 第二阶段：模块级架构收敛

目标：把最小原型变成稳定的流式系统。

建议工作项：

1. 为 FM-index 设计 DDR 分块布局。
2. 为 S-W 阵列引入候选队列和回压控制。
3. 为 rescue 设计独立调度入口，避免打乱主路径。
4. 为 PKPE 设计 BRAM/URAM 映射方案。
5. 为 GLCE 设计固定宽度和饱和逻辑。
6. 将各模块之间统一为 streaming interface。

建议的工程接口：

1. `input stream` : reads, qualities, pair metadata
2. `mapping output stream` : candidate alignments, scores, flags
3. `caller input buffer` : active regions or local reference windows
4. `variant stream` : discovered variants
5. `genotype output stream` : genotype + PL

这一阶段要解决的是系统稳定性，而不是论文指标。

9.4 第三阶段：引入论文关键优化

目标：把论文中最重要的性能思想逐步加入。

优先级建议如下：

1. 引入 RSC，先把 S-W 工作量压下来。
2. 引入 paired-end simultaneous alignment。
3. 引入 rescue 且让其计算尽量被主流水隐藏。
4. 把 caller 改为独立 engine。
5. 引入 ping-pong buffers。
6. 扩展 PKPE 并支持动态 k-mer。
7. 扩展 GLCE 并增加更多 likelihood 并行度。

这里最值回票价的是 RSC，因为它能直接减少 mapping 主瓶颈的 DP 压力。

9.5 第四阶段：端到端板级系统搭建

目标：形成完整 FPGA 板级原型。

建议板级数据流：

1. Host CPU 将 reads、reference、FM-index 预加载到 DDR。
2. FPGA 侧 DMA 读取数据到各模块。
3. mapping 结果写入中间缓冲。
4. caller 从中间缓冲读取局部区域数据继续处理。
5. 最终 genotype/PL 结果回写 DDR。
6. Host CPU 负责生成 VCF 或其他输出格式。

这与论文的系统分工是一致的，也更适合 FPGA 实际开发。

9.6 第五阶段：精度与性能共同验证

目标：验证不是“跑起来就算完成”，而是真正可用于论文级比较。

至少要做三类验证：

1. 功能验证
2. 精度验证

3. 性能验证

功能验证关注：

1. 每一级输出是否正确 °
2. 边界条件是否稳定 °

精度验证关注：

1. 与软件基线在 mapping 上的差异 °
2. rescue 是否确实提升 sensitivity °
3. 定点 GLCE 与浮点软件计算的偏差 °

性能验证关注：

1. 每阶段吞吐 °
2. 每阶段 stall 比例 °
3. DDR 带宽占用 °
4. PE 利用率 °
5. 端到端 latency 和 reads/s °

十、建议的模块拆分顺序

如果按研发优先级排序，我建议如下：

1. Smith-Waterman PE array
2. Viterbi/Pair-HMM PE array
3. FM-index exact matching
4. RSC
5. rescue 调度
6. variant discovery backtracking
7. de Bruijn graph / PKPE
8. 端到端 buffer 与 DMA
9. 双引擎并行化

原因很简单：

1. 真正的计算瓶颈是 DP °
 2. 先把最重的核做扎实 ’系统才有继续集成的价值 °
 3. graph 和 system integration 虽复杂 ’但如果 DP 核心吞吐没有底座 ’后面调度优化意义不大 °
-

十一 、最现实的实现目标

如果项目要收敛 ’建议把目标分成三档 °

11.1 保守目标

实现功能上完整的 FPGA 原型 :

1. 能从 reads 走到 genotype/PL °
2. 支持 paired-end °
3. 支持 rescue °
4. 输出结果与软件基线基本一致 °

11.2 中等目标

实现接近论文架构思想的系统 :

1. RSC + S-W
2. PKPE 风格 graph engine
3. GLCE 风格 Viterbi likelihood engine
4. caller/aligner 分离
5. ping-pong buffering

11.3 激进目标

追求论文量级吞吐和高能效 °

这一目标在 FPGA 上可以作为研究目标 ’但不应作为第一阶段承诺 ’因为它高度依赖 :

1. 板卡资源规模
2. 存储层次设计

3. 时钟收敛
 4. 大量微架构优化
-

十二、最终判断

综合全文分析，可以给出最终判断：

1. 论文提出的是一条明确、可硬件化的端到端 NGS 分析算法链。
2. 其关键模块都适合在 FPGA 上实现，尤其是 FM-index、S-W、Viterbi/Pair-HMM、固定结构 graph engine。
3. 论文已经给出了足够的高层算法与模块划分，足以支持设计一个功能上等价的 FPGA 原型。
4. 但论文没有提供足够细的微结构规格，无法支持无歧义复刻作者 ASIC。
5. 因此，最合理的工程目标是：以 FPGA 实现论文同类端到端算法系统，并逐步引入 RSC、rescue、PKPE、GLCE、ping-pong buffering 等关键优化。